

Scaling Simulator

Scaling Simulator Goals: **measure** how performance scales:

- Across multiple SMs
- Across multiple warps on a single SM
- With synchronization (barriers).

Basic structure of simulator: **6 stages**

- Fetch**: Schedule warp to execute and update scoreboard
- Decode+Execute**
- Memory** (rising and falling): send and receive memory requests
- Write Back**: updates scoreboard

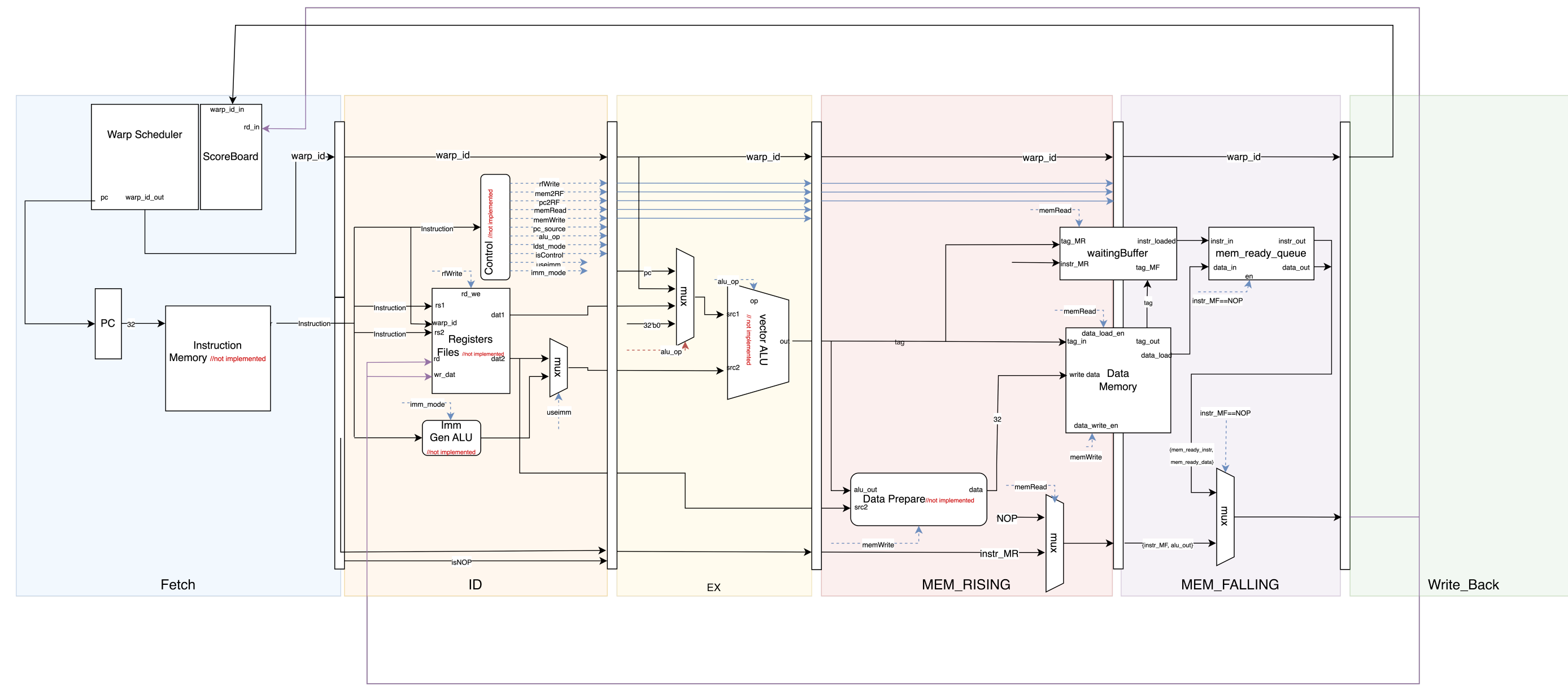


Figure 1. Diagram of the Scaling Simulator.

Divergence Simulator

Divergence Simulator Goals:

- Measure thread divergence** across different programs.
- Explore effectiveness of different strategies** used to manage divergence, for example, reconvergence schedulers.

To achieve **both** goals, simulator needs to support **computation**:

- The divergence simulator supports execution of a restricted subset of PTX (“PTX-Minus”).
- Simple transpiler translates NVIDIA PTX code to PTX-Minus.

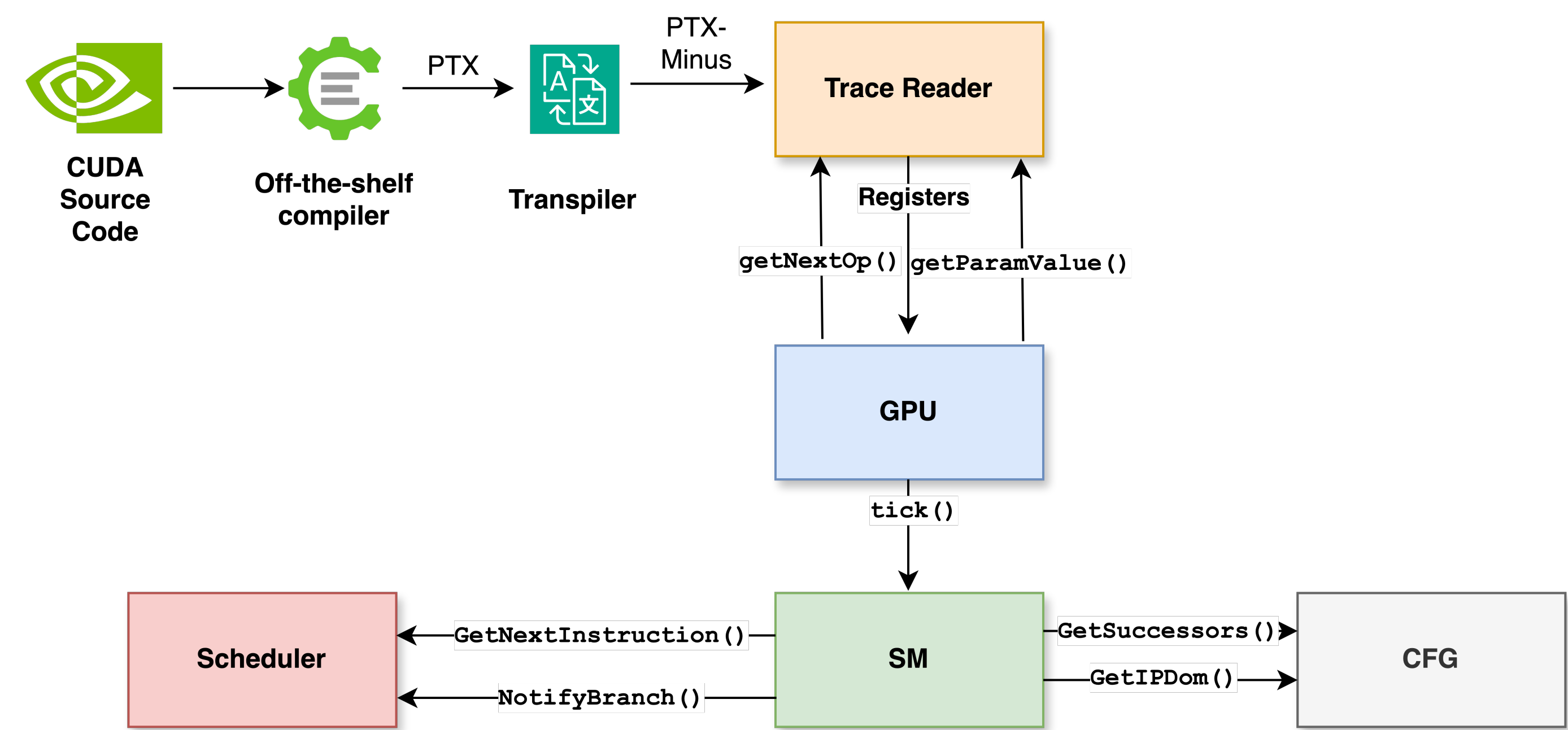


Figure 2. Architecture of the Divergence Simulator.

Scaling Simulator Results

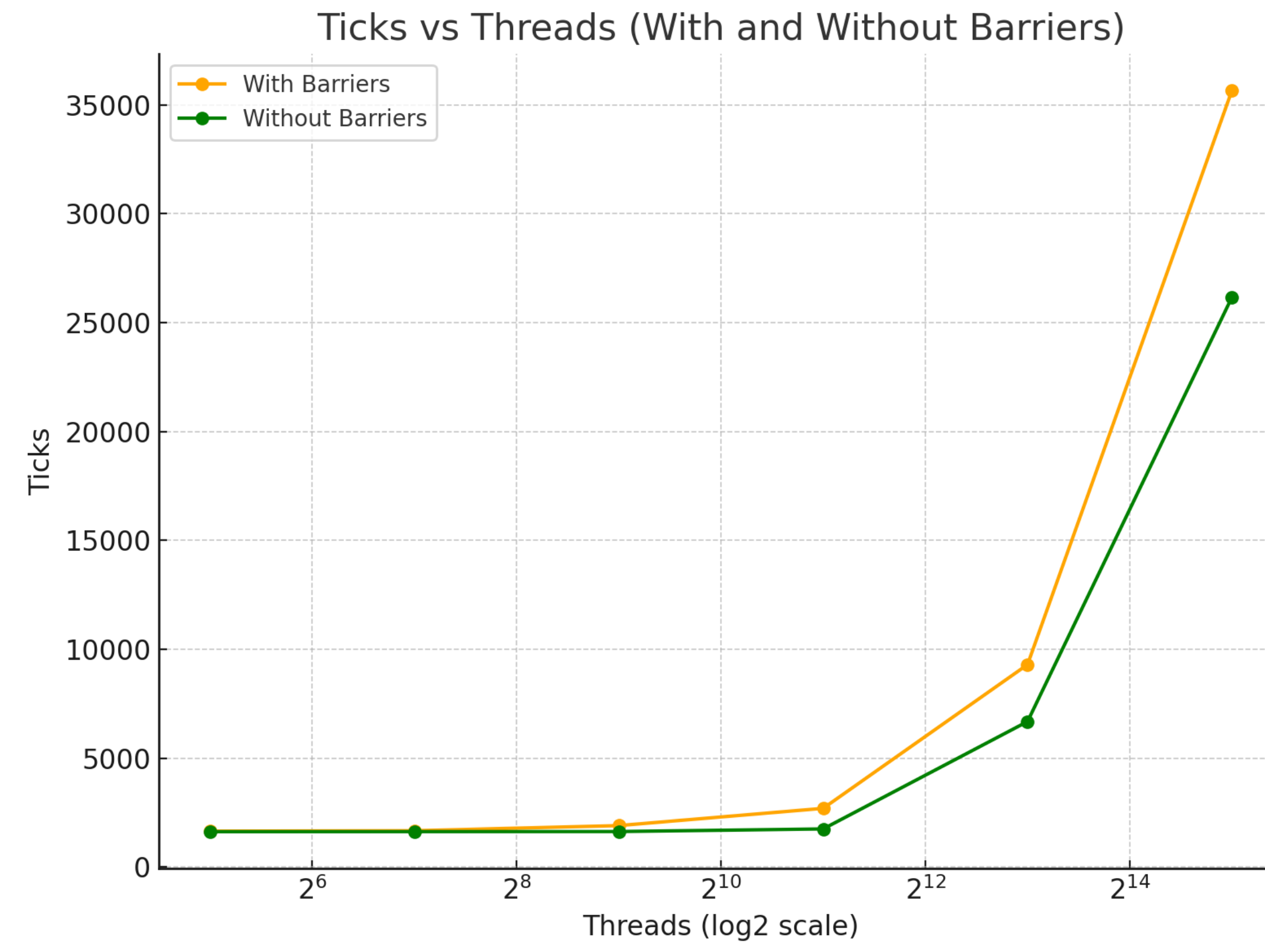


Figure 3. Runtime of memory intensive trace with respect to threads on 1 SM with and without barriers

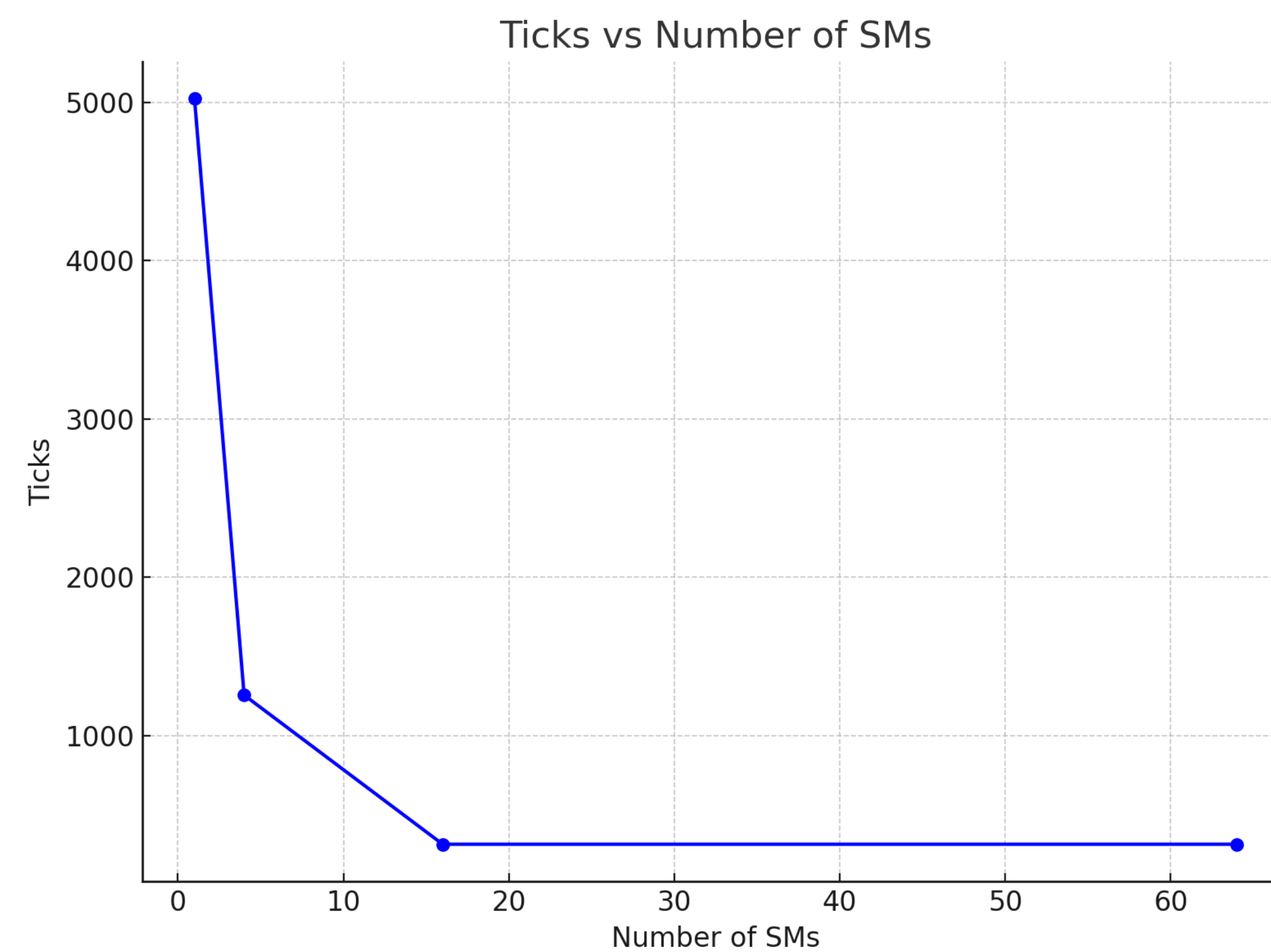


Figure 4. Runtime of a trace with 16 thread blocks compared to SM cores used

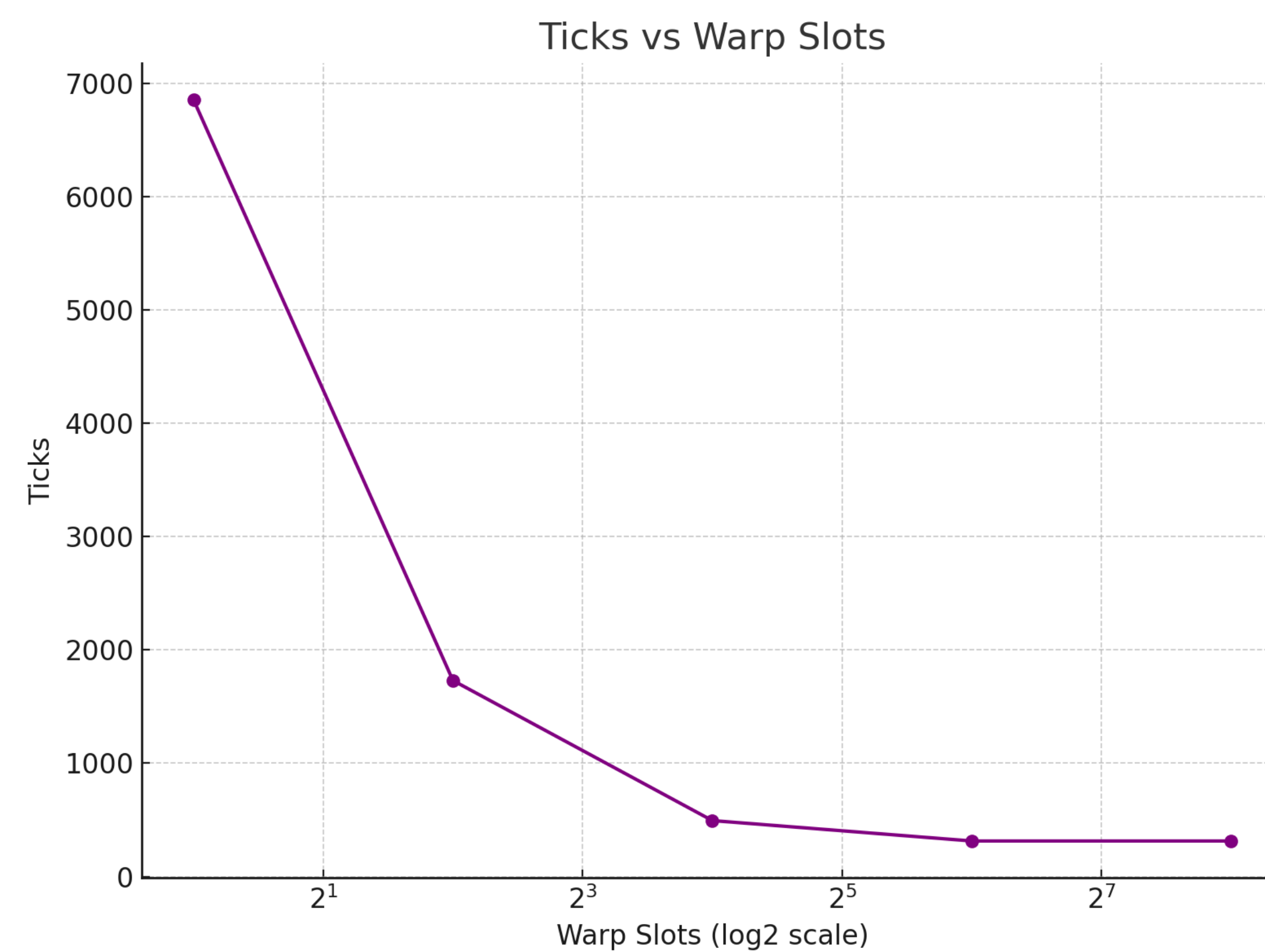


Figure 5. Runtime of a trace compared to different warp slots configurations SM, which is the maximum number of warps SM can run concurrently

Measuring Thread Divergence

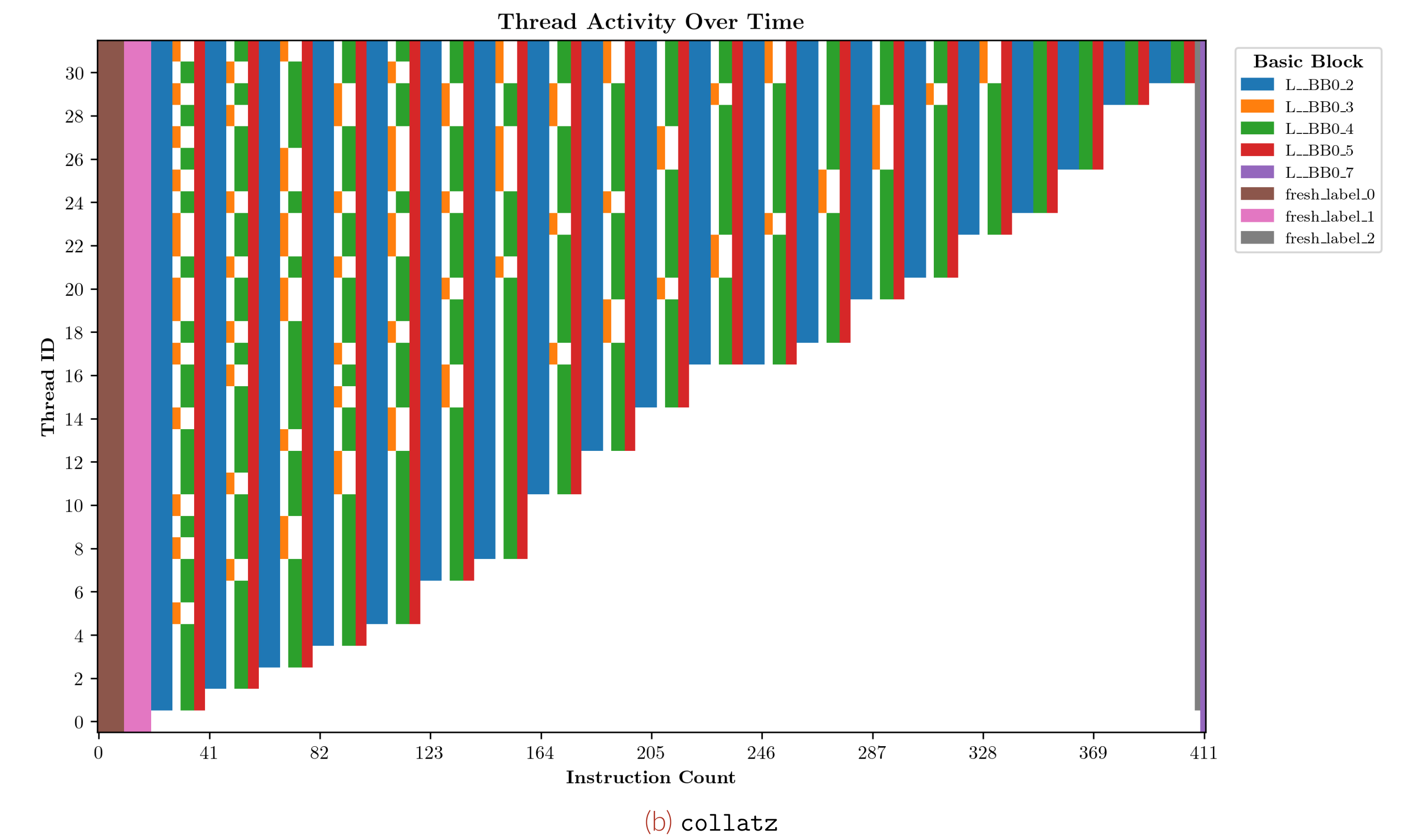
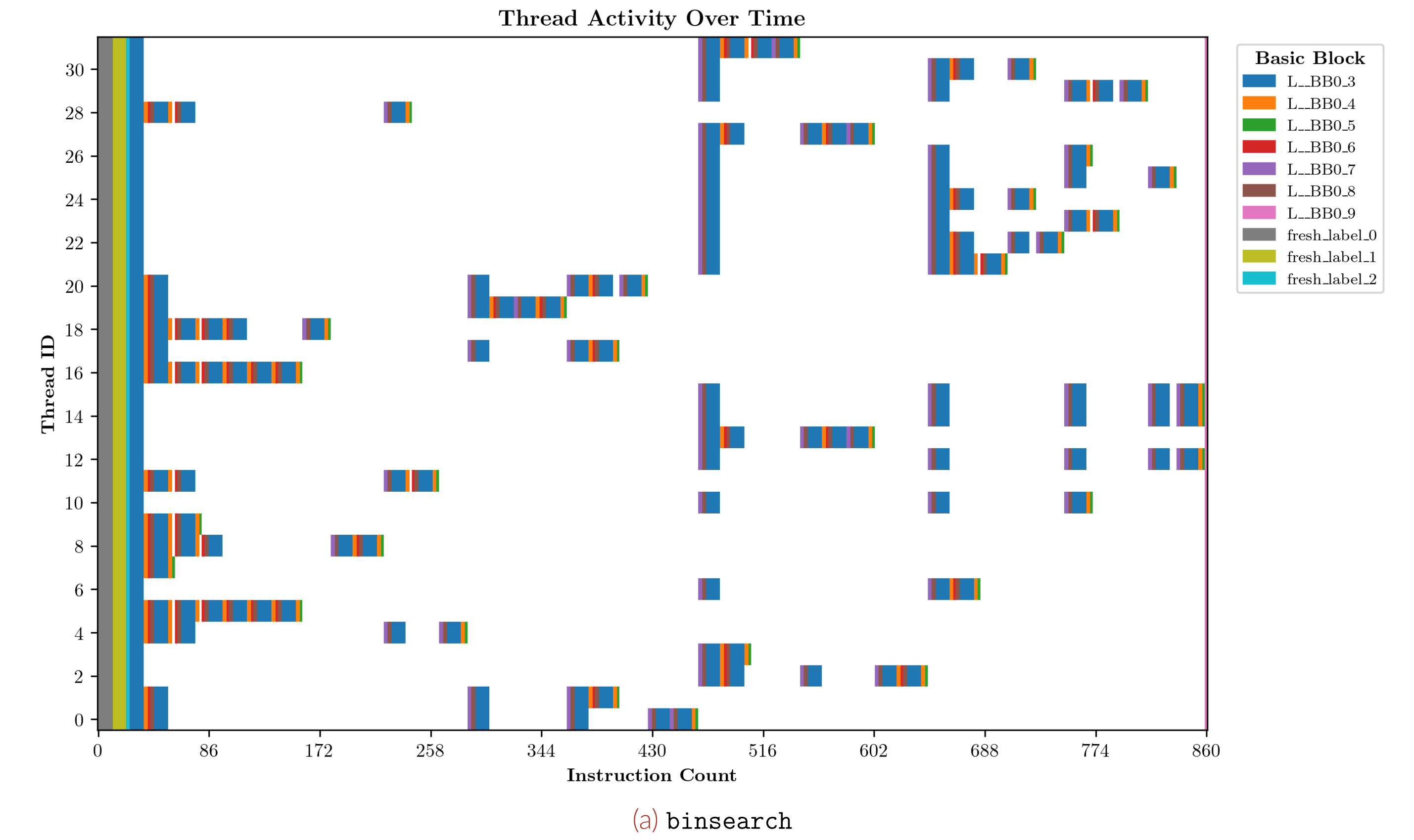


Figure 6. Heatmap representation for thread activity for one warp executing the **binsearch** and **collatz** example programs using the **Reconvergence Scheduler**. In each chart, the x -axis is instruction count, and the y -axis is the thread ID. A colored cell indicates that the thread is active for the relevant instruction, and a blank cell indicates that the thread is inactive. The specific color of each colored cell corresponds to the basic block the instruction belongs to.

Mitigating Thread Divergence

Two scheduler implementations evaluated:

- Naive**. Put branch targets on stack, pop only when top entry completely finishes execution.
- Reconvergence Stack**. Identify reconvergence points at compile time (immediate post-dominators). Runtime stack enables threads to reconverge at these points.

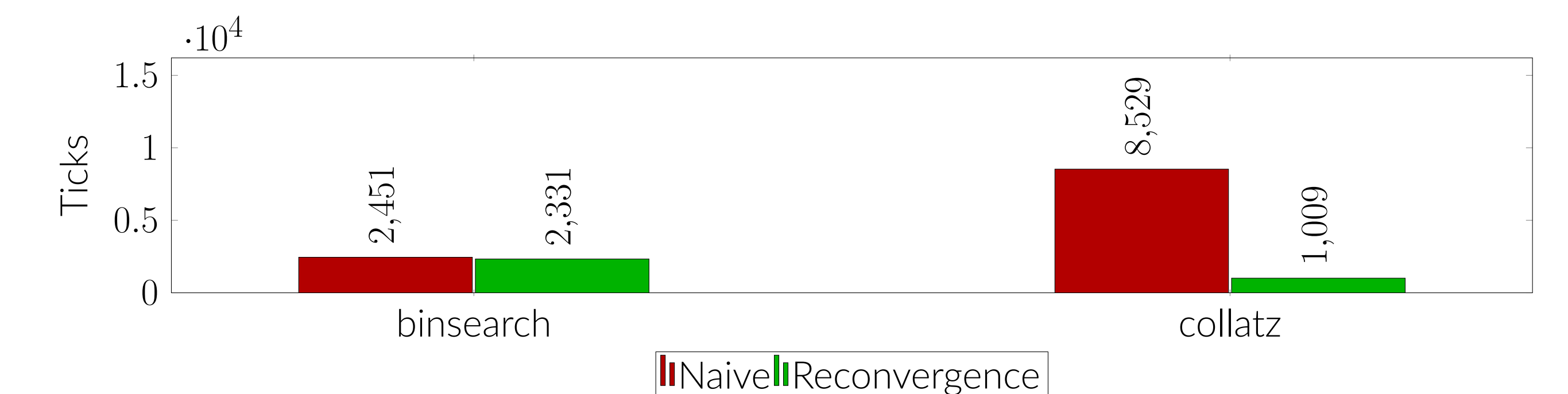


Figure 7. Runtime in Ticks for the *Naive* and *Reconvergence* strategies on the **binsearch** and **collatz** traces.